# CSA47 – DEEP LEARNING

# ASSIGNMENT – 2

## CO2 – DEEP FEED-FORWARD AND CONVOLUTIONAL NETWORK DESIGN

### Scenario: Smart Agriculture and Crop Disease Detection

An agricultural research centre wants to predict crop yield from rainfall, temperature, soil moisture and fertilizer quantity, and to classify healthy and diseased crop leaves from images. The answers below design, train, optimize and evaluate suitable deep neural network architectures.

### TASK 1 – Deep Feed-Forward Network Architecture

#### a) Network design

| Layer | Neurons | Role |
|---|---|---|
| Input | 4 | Rainfall, temperature, soil moisture, fertilizer quantity |
| Hidden 1 | 8 | Learns combinations of inputs (ReLU) |
| Hidden 2 | 4 | Learns higher-level combinations (ReLU) |
| Output | 1 | Predicted crop yield (linear activation, regression) |

The architecture is 4 – 8 – 4 – 1: a fully connected feed-forward network in which every neuron of one layer connects to every neuron of the next layer. The output layer has a single linear neuron because yield is a continuous value.

#### b) Dimensions of weight matrices and bias vectors

Using the convention $z = Wa + b$ with column vectors, W has shape (neurons in current layer × neurons in previous layer):

| Layer | Weight matrix | Bias vector | Parameters |
|---|---|---|---|
| Input → Hidden 1 | $W^1 : 8 \times 4$ | $b^1 : 8 \times 1$ | $32 + 8 = 40$ |
| Hidden 1 → Hidden 2 | $W^2 : 4 \times 8$ | $b^2 : 4 \times 1$ | $32 + 4 = 36$ |
| Hidden 2 → Output | $W^3 : 1 \times 4$ | $b^3 : 1 \times 1$ | $4 + 1 = 5$ |
| Total | | | **81** |

#### c) Forward propagation

Let $x \in \mathbb{R}^4$ be the normalized input vector. Then:

- Hidden layer 1: $z^1 = W^1x + b^1$ $(8 \times 1)$, $a^1 = \text{ReLU}(z^1)$
- Hidden layer 2: $z^2 = W^2a^1 + b^2$ $(4 \times 1)$, $a^2 = \text{ReLU}(z^2)$
- Output: $\hat{y} = W^3a^2 + b^3$ (scalar, linear activation)

In one expression: $\hat{y} = W^3 \cdot \text{ReLU}(W^2 \cdot \text{ReLU}(W^1x + b^1) + b^2) + b^3$. The inputs are multiplied by weights, summed with a bias, passed through the non-linear activation, and the result is fed forward layer by layer until the single output neuron gives the crop-yield prediction. The non-

linear activations allow the network to model non-linear relationships such as diminishing returns of fertilizer.

## TASK 2 – Forward Propagation and Activation Functions

### a) Weighted sum

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b = (0.4)(0.6) + (0.3)(0.8) + (0.2)(0.5) + 0.1$$

$$z = 0.24 + 0.24 + 0.10 + 0.10 = \mathbf{0.68}$$

### b) ReLU and interpretation

$$\text{ReLU}(z) = \max(0, z) = \max(0, 0.68) = \mathbf{0.68}$$

**Interpretation:** the weighted sum is positive, so the neuron is active and passes its value unchanged to the next layer (output 0.68). Had z been negative, ReLU would output 0 and the neuron would stay silent. ReLU introduces non-linearity cheaply and has gradient 1 for positive inputs, which helps training.

## TASK 3 – Gradient-Based Learning and Loss Calculation

### a) Need for a loss function and choice for regression

A loss function converts the difference between the prediction $\hat{y}$ and the true value y into a single number that measures how wrong the network is. Without it there is nothing to minimise, no gradient to compute and no way to tell whether a parameter change improves the model. For regression the suitable choice is **Mean Squared Error (MSE)**, $L = (1/N) \Sigma (y_i - \hat{y}_i)^2$. It is smooth and differentiable, penalises large errors strongly, and has a simple gradient.

### b) Squared error loss

$$L = (y - \hat{y})^2 = (50 - 42)^2 = 8^2 = \mathbf{64}$$

### c) Interpretation

The prediction is 8 units below the actual yield (16% under-prediction), giving a loss of 64 (RMSE = 8 units). A perfect prediction would give L = 0, so the value shows the network is still inaccurate. Because the error is squared, a miss of 16 would cost 256, four times as much, so large errors dominate the loss.

### d) Gradient-based learning

Training seeks parameters $\theta = \{W, b\}$ that minimise the loss. The steps are: (1) forward pass to compute $\hat{y}$; (2) compute the loss L; (3) use backpropagation to compute the gradient $\partial L/\partial \theta$, which says how L changes if each parameter changes; (4) update each parameter in the direction that reduces the loss, $\theta \leftarrow \theta - \eta\, \partial L/\partial \theta$, where $\eta$ is the learning rate; (5) repeat over many iterations (epochs) until the loss stops decreasing. Here, $\partial L/\partial \hat{y} = -2(y - \hat{y}) = -16$, a negative value showing that increasing $\hat{y}$ would reduce the loss.

## TASK 4 – Gradient Descent and Weight Update

### a) Updated weight

$$w_new = w - \eta\, (\partial L/\partial w) = 0.5 - (0.1)(0.8) = 0.5 - 0.08 = \mathbf{0.42}$$

#### b) Why the update opposes the gradient

The gradient points in the direction in which the loss increases most steeply. To reduce the loss we must move the opposite way, so we subtract the gradient. Here the gradient is positive (increasing w increases the loss), so w is decreased from 0.5 to 0.42. Mathematically, a small step $\Delta w = -\eta\, g$ gives $\Delta L \approx g \cdot \Delta w = -\eta\, g^2 \leq 0$, so the loss decreases.

#### c) Effect of the learning rate

- **Extremely small η:** very small steps, extremely slow convergence, high training cost, and a risk of getting stuck on plateaus or in poor local minima.
- **Extremely large η:** steps overshoot the minimum, the loss oscillates or even increases (divergence), and weights can overflow (exploding values).
- **Appropriate η:** steady, fast decrease of loss, often combined with a decaying schedule.

### TASK 5 – Back-Propagation Through Multiple Layers

#### a) How backpropagation assigns responsibility for the error

After the forward pass, the error at the output ($\hat{y} - y$) is propagated backwards layer by layer. At each layer we compute an error signal $\delta$ that tells how much that layer's pre-activation contributed to the loss, and from it the gradient of every weight ($\partial L/\partial W = \delta \cdot a^T$). Weights that strongly influenced the wrong prediction receive large gradients and are changed more; weights with little influence receive small gradients.

#### b) Role of the chain rule

The loss depends on early-layer weights only through a chain of intermediate functions, so the gradient is a product of local derivatives. For the three-layer network with $L = (y - \hat{y})^2$:

- Output: $\delta^3 = \partial L/\partial z^3 = -2(y - \hat{y})$
- Hidden 2: $\delta^2 = (W^{3T}\, \delta^3) \odot f'(z^2)$
- Hidden 1: $\delta^1 = (W^{2T}\, \delta^2) \odot f'(z^1)$
- Gradients: $\partial L/\partial W^3 = \delta^3 (a^2)^T$, $\partial L/\partial W^2 = \delta^2 (a^1)^T$, $\partial L/\partial W^1 = \delta^1 x^T$, and $\partial L/\partial b\hat{}l = \delta\hat{}l$

The chain rule lets each layer reuse the error already computed for the layer above, so all gradients are obtained in a single backward pass at a cost similar to the forward pass.

#### c) Using gradients to update weights

Once the gradients are known, every weight and bias is updated by gradient descent (or a variant such as Adam): $W\hat{}l \leftarrow W\hat{}l - \eta\, \partial L/\partial W\hat{}l$ and $b\hat{}l \leftarrow b\hat{}l - \eta\, \partial L/\partial b\hat{}l$. Repeating the forward pass, backward pass and update for many batches reduces the gap between predicted and actual yield.

### TASK 6 – Activation Functions and Vanishing/Exploding Gradients

#### a) How activation functions cause vanishing gradients

By the chain rule, the gradient reaching early layers is a product of many factors of the form $W \cdot f'(z)$. Sigmoid has derivative $f'(z) = \sigma(z)(1 - \sigma(z)) \leq 0.25$, and it is almost 0 when $|z|$ is large (saturation). Multiplying many numbers below 1 shrinks the gradient exponentially: for 10 sigmoid layers, $0.25^{10} \approx 10^{-6}$. Early layers therefore receive almost no signal and learn very slowly. If the factors are above 1, the opposite happens and gradients explode.

**b) Comparison for hidden layers**

| Function | Range | Max derivative | Remarks |
|---|---|---|---|
| Sigmoid | (0, 1) | 0.25 | Saturates, not zero-centred, strong vanishing gradient; suited to output of binary classification only |
| Tanh | (−1, 1) | 1.0 | Zero-centred, better than sigmoid, but still saturates for large \|z\| |
| ReLU | [0, ∞) | 1 (for z > 0) | No saturation for positive z, cheap, sparse activation; can "die" if z stays negative (Leaky ReLU/ELU fix this) |

**c) Impact on deep-network training**

Activation choice controls gradient magnitude and therefore training speed and stability. ReLU-type activations keep the gradient at 1 for active units, enabling deep networks to train faster and reach better solutions. Further remedies for the slow-learning crop network: use ReLU/Leaky ReLU in hidden layers, He initialization, batch normalization, residual (skip) connections and gradient clipping against explosion.

## TASK 7 – Back-Propagation and Optimization Strategy

**a) Effect of the learning rate**

The learning rate sets the step size of each update. Too small: the loss decreases slowly (first experiment). Too large: the steps overshoot the minimum, so the loss decreases quickly at first but fluctuates strongly or diverges (second experiment). A well-chosen rate gives a fast, smooth decrease.

**b) Backpropagation and gradient descent in one iteration**

1. Take a mini-batch of training samples.
2. **Forward pass:** compute predictions $\hat{y}$ and the loss L.
3. **Backpropagation:** compute $\partial L/\partial \theta$ for all weights and biases using the chain rule.
4. **Gradient descent:** update $\theta \leftarrow \theta - \eta\, \partial L/\partial \theta$.
5. Move to the next mini-batch. One full pass through the data is one epoch.

Backpropagation supplies the gradient; gradient descent uses it to change the parameters.

**c) Suggested improvements**

- **Slow decrease:** increase the learning rate (e.g., learning-rate range test), use momentum or Adam, normalize inputs, add batch normalization, use ReLU and He initialization, use a learning-rate warm-up.
- **Fast but fluctuating:** reduce the learning rate, apply learning-rate decay (step, exponential or cosine), increase the batch size to reduce gradient noise, use momentum/Adam, and clip gradients.

**d) Why monitor the loss curve**

The curves of training and validation loss reveal the training state: a flat curve suggests a too small learning rate or vanishing gradients; oscillation or spikes suggest too large a learning

rate; training loss falling while validation loss rises indicates overfitting (stop early or regularise); both high indicates underfitting. Monitoring allows timely tuning, early stopping and checkpoint selection.

## TASK 8 – Regularization and Overfitting Control

### a) Learning problem and evidence

**Overfitting (high variance).** Training accuracy is 99% while validation accuracy is only 78%, a generalization gap of 21 points. The network has memorised the training data, including noise, instead of learning patterns that transfer to unseen data.

### b) How L1 and L2 control complexity

- **L2 (weight decay):** adds $\lambda\Sigma w^2$ to the loss, penalising large weights. Weights shrink smoothly towards zero, the function becomes smoother and less sensitive to noise.
- **L1:** adds $\lambda\Sigma|w|$ to the loss. It pushes many weights to exactly zero, producing sparse networks and implicit feature selection.

Both limit the effective capacity of the model, so that variance is reduced at the cost of a slight increase in bias.

### c) Regularized loss function

$$L_total = (1/N)\ \Sigma_i\ (y_i - \hat{y}_i)^2 + \lambda\ \Sigma_l\ \|W^l\|^2_F = L_data + \lambda\ \Sigma\ w^2$$

$\lambda \geq 0$ is the regularization parameter controlling the trade-off between fitting the data and keeping weights small. $\lambda = 0$ means no regularization (overfitting risk); a very large $\lambda$ forces weights towards zero (underfitting). It is chosen by validation (e.g., $10^{-5}$ to $10^{-2}$). The gradient becomes $\partial L/\partial w + 2\lambda w$, so each update also shrinks the weight (weight decay).

### d) Dropout

During training, each hidden neuron is randomly set to zero with probability p (e.g., 0.2–0.5) in every iteration. Neurons cannot rely on specific co-adapted partners, so each must learn robust, independent features. It is equivalent to training an ensemble of many thinned networks. At test time all neurons are used, with activations scaled (or inverted dropout during training), which gives better generalization and a smaller gap between training and validation accuracy. Early stopping, data augmentation and more data also help.

## TASK 9 – Convolution Operation and Feature Extraction

### a) Why convolutional layers suit images

- **Local connectivity:** each neuron looks at a small patch, ideal for local patterns such as spots, edges and textures.
- **Weight sharing:** the same filter slides over the whole image, so a lesion is detected wherever it appears (translation equivariance) and the number of parameters is small.
- **Hierarchical features:** stacked layers build edges → textures → lesion shapes → whole-leaf patterns.
- A fully connected layer would need a weight for every pixel pair, ignoring spatial structure and requiring millions of parameters.

### b) Convolution example

Input image (4 × 4) and a 3 × 3 vertical-edge filter, stride 1, no padding:

| Image | | | |
|---|---|---|---|
| 1 | 2 | 0 | 1 |
| 3 | 1 | 1 | 0 |
| 0 | 2 | 1 | 2 |
| 1 | 0 | 3 | 1 |

Filter K = [[1, 0, −1], [1, 0, −1], [1, 0, −1]]. Each output value is the sum of element-wise products of the filter with the image patch (equivalently, left column sum minus right column sum):

- Position (1,1): (1+3+0) − (0+1+1) = 4 − 2 = **2**
- Position (1,2): (2+1+2) − (1+0+2) = 5 − 3 = **2**
- Position (2,1): (3+0+1) − (1+1+3) = 4 − 5 = **−1**
- Position (2,2): (1+2+0) − (0+2+1) = 3 − 3 = **0**

Feature map = [[2, 2], [−1, 0]]. Output size = (4 − 3)/1 + 1 = 2, so the 4 × 4 image becomes a 2 × 2 map.

**Interpretation:** a large positive value means the left side of the patch is brighter than the right side, i.e. a vertical edge exists there (e.g., the border of a lesion); a negative value indicates the opposite transition; 0 means no vertical change. The map highlights where the feature is located.

### c) Roles of filters, stride and padding

- **Filters (kernels):** small learnable weight matrices that detect patterns such as edges, spots or texture; many filters give many feature maps (channels).
- **Stride:** the step size of the filter. Stride 1 keeps detail; larger strides downsample and reduce computation.
- **Padding:** adding zeros around the border. “Same” padding keeps the output the same size and preserves border information; “valid” (no padding) shrinks the map. Output size = (n + 2p − f)/s + 1.

## TASK 10 – CNN Architecture and Pooling

### a) Function of each component

| Component | Function |
|---|---|
| Input image | Raw leaf image (e.g., 32 × 32 × 3 RGB) |
| Convolution | Applies learnable filters to extract local features (edges, spots, texture) |
| ReLU | Adds non-linearity (max(0, x)); avoids saturation and speeds training |
| Pooling | Downsamples feature maps, reducing size and computation and adding small-shift invariance |

| Component | Function |
|---|---|
| Second Conv + ReLU + Pool | Combines low-level features into higher-level patterns |
| Fully connected layer | Flattens features and combines them to make the classification decision |
| Output | Softmax (or sigmoid) probabilities for healthy vs. diseased |

#### b) How pooling reduces dimensions

Pooling slides a window (e.g., 2 × 2, stride 2) over each feature map and keeps one value per window (maximum for max pooling, mean for average pooling). Example of 2 × 2 max pooling on a 4 × 4 map:

$$[[1, 3, 2, 1], [4, 6, 5, 2], [7, 2, 8, 3], [1, 9, 4, 6]] \rightarrow [[6, 5], [9, 8]]$$

The size falls from 4 × 4 to 2 × 2 (75% fewer values), the number of channels is unchanged, and the strongest responses are kept. Pooling has no learnable parameters.

#### c) Effect of filter size, stride and number of filters

Output size = $\lfloor (n + 2p - f)/s \rfloor + 1$. Example: a 32 × 32 × 3 input with six 5 × 5 filters, stride 1, no padding gives 28 × 28 × 6; pooling gives 14 × 14 × 6; a second convolution with sixteen 5 × 5 filters gives 10 × 10 × 16; pooling gives 5 × 5 × 16 = 400 features for the fully connected layer.

- **Filter size:** larger filters see more context but cost more parameters and shrink the map more; 3 × 3 is the common choice.
- **Stride:** larger stride gives smaller maps and less computation but loses detail.
- **Number of filters:** determines the depth (channels) of the output; more filters capture more feature types, with more parameters (conv1 here: $(5 \cdot 5 \cdot 3 + 1) \cdot 6 = 456$; conv2: $(5 \cdot 5 \cdot 6 + 1) \cdot 16 = 2{,}416$).

#### d) Hierarchical feature learning

Early layers respond to simple patterns (edges, colour blobs). Because each later layer sees the output of the previous one, its receptive field grows and it combines simple features into more complex ones: textures, spots and vein patterns, then lesion shapes and finally disease-specific patterns of the whole leaf. The fully connected layer uses these high-level features to classify healthy or diseased.

### TASK 11 – Comparative Evaluation of Feed-Forward and CNN Models

#### a) Why Model A suits structured numerical data

Weather and soil variables are a small, fixed-length vector of numbers with no spatial arrangement. A feed-forward network can learn non-linear interactions between features (e.g., rainfall × fertilizer) directly, is light to train, and works well on tabular data of modest dimensionality.

#### b) Why Model B suits images

Images have a spatial structure: neighbouring pixels are correlated and the same pattern (a spot or edge) can appear anywhere. CNNs exploit this by local filters, weight sharing and pooling, giving translation-robust features with far fewer parameters than a fully connected network on raw pixels.

#### c) Comparison

| Aspect | Model A (Feed-forward) | Model B (CNN) |
|---|---|---|
| Input representation | Vector of 4 numerical features | Tensor H × W × 3 (pixels), e.g., 224 × 224 × 3 |
| Feature extraction | Manual feature design/normalization; layers mix all inputs | Learned automatically by filters (edges → textures → objects) |
| Parameter organization | Dense matrices W (each weight used once) | Small shared filters reused over the image; sparse connectivity |
| Parameters (example) | 81 for the 4–8–4–1 network | Conv 3×3×3×64 = 1,728 versus ~150 million for a dense layer of 1,000 units on a 224×224×3 image |
| Computation | Very low; CPU is sufficient | High (convolutions on large tensors); GPU needed |

#### d) Recommendation

- **Yield prediction:** Model A (feed-forward regression with MSE loss, ReLU, L2/dropout), because the data are numerical, low-dimensional and cheap to process.
- **Leaf disease classification:** Model B (CNN, preferably with transfer learning), because the data are images and spatial patterns are decisive.

Both can be combined later (e.g., image-based disease probability as an additional input to the yield model).

### TASK 12 – Design Challenge: Optimize and Redesign the Deep Learning System

**Decision:** redesign. Keep a (regularised) feed-forward network for the 500,000 numerical records and add a CNN branch for the 100,000 leaf images, with a better optimization and regularization strategy.

#### A. Impact Analysis

#### a) Effect of larger data and added images

- **Numerical data (10,000 → 500,000 records):** 50× more data lowers variance and supports a deeper model, but full-batch gradient descent becomes impractical, so mini-batch training (e.g., batch size 128–512) and efficient data pipelines are needed. Epoch cost grows 50×.
- **Leaf images (100,000):** each image has tens to hundreds of thousands of values (224 × 224 × 3 = 150,528), so the data volume (about 15 GB as uint8 at 224 × 224) requires GPU training, streaming/batched loading, data augmentation and a convolutional architecture. Memory and compute for activations and gradients rise sharply.

- **Architecture:** two data types (tabular and image) need different networks (MLP and CNN) and possibly a fusion stage.

**b) Limitations of a feed-forward network on image data**

- Flattening destroys spatial structure; neighbouring pixels are treated as unrelated inputs.
- Parameter explosion: 150,528 inputs × 1,000 units ≈ 150 million weights in the first layer alone, giving high memory use, slow training and strong overfitting.
- No translation invariance: a lesion in a new position must be learned again.
- No weight sharing and no hierarchical feature extraction, hence poor accuracy.

### B. Evaluation of the Existing Design

**a) Evaluation:** high training accuracy but lower validation performance indicates overfitting (high variance): the model fits training data but generalises poorly. Long training time shows inefficient optimization (e.g., small batches or full-batch updates, poor learning rate, no early stopping) at the new data scale. The model also has no way to use the image data.

**b) Is more depth and neurons the best solution?** No. Larger capacity lowers bias but increases variance, so the generalization gap would widen, and training time and memory would grow. Deeper plain networks also suffer from vanishing gradients. Better remedies are regularization (L2, dropout, early stopping), more data (now available), batch normalization, a better optimizer and learning-rate schedule, and a suitable architecture (CNN) for images. Capacity should only be increased if validation tests show underfitting.

### C. Adaptation or Redesign

**a) Proposed architectures**

- **Numerical crop data (regression):** input 4 (or more engineered features) → Dense 64 (BatchNorm, ReLU, Dropout 0.2) → Dense 32 (BatchNorm, ReLU, Dropout 0.2) → Dense 1 (linear); MSE loss; L2 weight decay.
- **Leaf images (classification):** 224 × 224 × 3 input → pretrained CNN backbone (ResNet-18/50 or EfficientNet-B0), fine-tuned: Conv–BN–ReLU blocks with pooling → Global Average Pooling → Dropout 0.3–0.5 → Dense (softmax: healthy/diseased, or per disease class); cross-entropy loss.

**b) Optimization and regularization methods**

- **Optimizer:** Adam (lr $10^{-3}$ for the MLP, $10^{-4}$ for fine-tuning the CNN) for fast convergence; SGD with momentum 0.9 is a strong alternative for final CNN training and generalisation.
- **Learning-rate adjustment:** warm-up followed by cosine decay, or ReduceLROnPlateau.
- **Regularization:** L2 weight decay ($10^{-4}$), dropout, early stopping on validation loss, data augmentation for images.
- **Batch normalization** after dense/conv layers for stable gradients and faster training.
- **Activations:** ReLU/Leaky ReLU in hidden layers, linear output for regression, softmax/sigmoid for classification; He initialization.

### D. Evidence-Based Defence

## a) Experimental design

- Use identical train/validation/test splits (e.g., 70/15/15, stratified for classes) and several random seeds to compare the **original** system (feed-forward only, 10,000 records) with the **redesigned** system (MLP + CNN).
- **Regression metrics (yield):** MSE, RMSE, MAE and $R^2$ on the test set; training vs. validation loss gap.
- **Classification metrics (leaf):** accuracy, precision, recall (sensitivity to diseased leaves), F1-score, confusion matrix and ROC-AUC.
- **Efficiency:** training time per epoch and to convergence, inference time, memory and number of parameters.
- **Diagnostics:** learning curves, ablation (with/without dropout, L2, batch norm, augmentation, transfer learning).

| Measure | Original | Redesigned (expected) |
|---|---|---|
| Training vs. validation gap | Large (overfitting) | Small |
| Validation RMSE / F1 | Higher RMSE; images not handled | Lower RMSE; F1 high on images |
| Training time to convergence | Long | Shorter (mini-batch, Adam, schedule, early stopping) |

**b) Justification:** the expected evidence is a smaller generalization gap, better validation RMSE and F1, and shorter convergence time. The original system cannot process images and overfits, so it should be **redesigned**: the numerical model can be retained in modified form (regularised, batch-normalised and optimized), and a CNN is added for images. Mathematically, regularization lowers variance, normalization and Adam improve conditioning and convergence, and convolution reduces parameters by weight sharing, so generalization and computational cost both improve.

### Additional design considerations

## Comparison of optimizers

| Method | Idea | Pros | Cons |
|---|---|---|---|
| SGD (batch size 1) | Update per sample | Cheap per step, escapes shallow minima | Very noisy, slow in wall-clock time |
| Mini-batch SGD | Update per batch of 32–512 | Good compromise, vectorised, GPU friendly | Needs lr tuning; sensitive to scaling |
| Momentum | $v \leftarrow \beta v + g$; $w \leftarrow w - \eta v$ | Smooths noise, speeds along consistent directions | Extra hyperparameter, may overshoot |
| RMSProp | Divides step by running RMS of gradients | Adapts per-parameter rates, good for non-stationary problems | No momentum or bias correction |
| Adam | Momentum + RMSProp with bias correction | Fast, robust defaults ($\beta_1$ 0.9, $\beta_2$ 0.999) | Can generalise slightly worse than tuned SGD |

**Learning-rate schedule:** linear warm-up for the first 3–5 epochs ($0 \rightarrow \eta_{max}$, e.g., $10^{-3}$), then cosine decay to about $10^{-6}$ by the final epoch: $\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\pi t/T))$; fall back to ReduceLROnPlateau (factor 0.1, patience 3) and early stopping (patience 5–10).

**Label-preserving image augmentation:** random horizontal/vertical flips and rotations (leaf orientation does not change disease), random crops and zoom (80–100%), small brightness/contrast changes (±10–20%) to cover lighting changes, slight Gaussian noise or blur, and mild translation. Avoid transformations that change the label, such as strong colour or hue shifts (yellowing or browning indicates disease), heavy cropping that removes the lesion, or cut-and-paste overlays. Augmentation is applied only to the training set.

**Comparison of image architectures**

| Model | Parameters | Typical input | Remarks |
|---|---|---|---|
| LeNet-5 | ~60 K | 32 × 32 gray | Tiny, fast; too weak for detailed leaf textures |
| AlexNet | ~61 M | 227 × 227 | First deep CNN with ReLU/dropout; large dense layers, heavy and overfit-prone |
| ResNet-50 | ~25.6 M | 224 × 224 | Skip connections solve vanishing gradients; strong accuracy, moderate cost |
| EfficientNet-B0 | ~5.3 M | 224 × 224 | Compound scaling, best accuracy per parameter and FLOP (~0.4 GFLOPs) |

**Recommendation:** with 100,000 images at about 224 × 224 and a limited compute budget, use **EfficientNet-B0 or ResNet-18/50 with ImageNet pretraining and fine-tuning**. LeNet is too small and AlexNet is inefficient. Pretrained weights reduce training time and need less data, and the residual or compound-scaling design keeps gradients healthy.